



The Vect Language Guide

A compiled language where scientific computing is syntax, not a library

Version 0.1.0

138 Tests Passing

Python 3.9 – 3.12

LLVM Native Codegen

By Eddie · github.com/Eddiegah/Vect · 2025

Table of Contents

1. What is Vect?
2. Installation & Setup
3. Your First Vect Program
4. The REPL — Interactive Mode
5. Variables & Types
6. Operators & Arithmetic
7. Control Flow — if, while, for
8. Functions
9. Vectors — Native First-Class Values
10. Matrices — Native First-Class Values
11. Symbolic Differentiation — The Headline Feature
12. Built-in Functions
13. String Operations
14. Type System & Error Messages
15. CLI Reference
16. Complete Example Programs
17. How the Compiler Works
18. Quick Reference Card

1. What is Vect?

Vect is a **compiled programming language** designed around a single idea: scientific computing should be part of the language itself, not something you bolt on with library imports.

In Python you write:

```
import numpy as np
import sympy as sp

v = np.array([1, 2, 3])
x = sp.Symbol('x')
f = x**2 + 3*x
df = sp.diff(f, x)
print(float(df.subs(x, 2)))
```

In Vect you write:

```
var v = [1, 2, 3]

sym f(x) = x**2 + 3*x
print(eval(d/dx(f(x)), x=2.0)) # → 7.0
```

`d/dx` is not a function call — it is a **language operator**, like `+` or `*`. `[1, 2, 3]` is a vector literal, as native as the number `42`. There are no imports. No library names. Just the language.

KEY FACT

Vect compiles to real native machine code via LLVM — the same compiler backend used by Clang, Rust, and Swift. It is not an interpreter and not a transpiler. Your program becomes actual x86-64 binary instructions.

Who is Vect for?

- Anyone curious about how programming languages and compilers work
- Students who want to see scientific computing treated as a first-class citizen
- Developers who want to explore a clean, readable language with minimal boilerplate
- Anyone who has ever wondered what it would feel like if `d/dx` was just... syntax

What can Vect do right now?

Feature	Status
Variables, arithmetic, comparisons, logic	✓ Full support
if / else if / else, while, for, break, continue	✓ Full support
Functions with parameters, return types, recursion	✓ Full support
Native vector literals and operations	✓ Full support
Native matrix literals and operations	✓ Full support
Symbolic differentiation with d/dx	✓ Full support
Interactive REPL	✓ Full support
Static type checker with helpful errors	✓ Full support
VS Code syntax highlighting	✓ Included
Multi-file imports	▣ Future work

2. Installation & Setup

BEFORE YOU START

Vect requires Python 3.9–3.12. Python 3.13 and above are

NOT

yet supported because `llvmlite 0.43.0` does not support them. Python 3.11 is the recommended version. No C++ compiler, no Visual Studio, and no CMake are needed.

Step 1 — Check your Python version

```
# Windows
py -0

# macOS / Linux
python3 --version
```

You need to see `3.9` , `3.10` , `3.11` , or `3.12` . If you only have 3.13+, install Python 3.11 from **python.org** alongside it.

Step 2 — Clone the repository

```
git clone https://github.com/Eddiegah/Vect.git
cd Vect
```

Step 3 — Create a virtual environment

```
# Windows (recommended)
py -3.11 -m venv venv
venv\Scripts\activate

# macOS / Linux
python3.11 -m venv venv
source venv/bin/activate
```

WHAT IS A VIRTUAL ENVIRONMENT?

It is an isolated Python installation just for this project. It keeps Vect's dependencies separate from everything else on your machine. Always activate it before using vect.

Step 4 — Install dependencies

```
pip install -r requirements.txt
pip install -e .
```

This installs four packages:

Package	Version	What it does
<code>llvmlite</code>	0.43.0	Python bindings to LLVM — the JIT compiler engine
<code>sympy</code>	1.13.1	Symbolic math — powers the <code>d/dx</code> operator behind the scenes
<code>click</code>	8.1.7	CLI framework for the <code>vect</code> command
<code>pytest</code>	8.2.2	Test runner

Step 5 — Verify the installation

```
python -c "import llvmlite.binding as llvm; llvm.initialize(); llvm.initialize_native_target(); llvm
```

You should see:

```
LLVM backend OK
```

If you see an error here, it almost always means you used the wrong Python version. Run `python --version` inside your venv and confirm it shows 3.11.x.

Step 6 — Run your first program

```
# Windows
venv\Scripts\vect run examples\demo.vect

# macOS / Linux
venv/bin/vect run examples/demo.vect
```

You should see output like:

```
--- 1. Functions & Recursion ---
55
--- 2. Native Vectors ---
[4, 1, 4, 2, 5]
4.0
--- 3. Matrix Multiply ---
[
  [0],
  [1]
]
--- 4. Symbolic Differentiation ---
3*x**2 - 4*x + 1
0.0
5.0
Done.
```

WINDOWS SHORTCUT

Instead of steps 3–6, you can run the one-shot setup script from the project folder:

```
.\setup_vect.ps1
```

Install VS Code Syntax Highlighting (optional but recommended)

```
# Windows – run in PowerShell from the Vect project folder
Copy-Item -Recurse vscode-extension "$env:USERPROFILE\.vscode\extensions\vect-lang-0.1.0"

# macOS / Linux
cp -r vscode-extension ~/.vscode/extensions/vect-lang-0.1.0
```

Restart VS Code, open any `.vect` file, and it will be coloured automatically.

3. Your First Vect Program

Create a file called `hello.vect` anywhere on your computer and type this:

```
# hello.vect - my first Vect program
```

```
print("Hello from Vect!")
```

```
var name = "Eddie"
```

```
print("Written by: " + name)
```

```
var x = 10
```

```
var y = 3
```

```
print(x + y)    # 13
```

```
print(x * y)    # 30
```

```
print(x ** 2)   # 100 - x squared
```

Run it:

```
venv\Scripts\vect run hello.vect
```

Output:

```
Hello from Vect!
```

```
Written by: Eddie
```

```
13
```

```
30
```

```
100
```

THINGS TO NOTICE

Comments start with # and run to the end of the line. `print()` is a built-in that accepts any type — integers, floats, strings, booleans, vectors, or matrices. There are no semicolons. Newlines end statements.

Your second program — using everything at once


```
# showcase.vect

# --- Vectors ---
var v1 = [1.0, 2.0, 3.0]
var v2 = [4.0, 5.0, 6.0]
print(v1 + v2)          # [5, 7, 9]
print(v1 · v2)          # 32.0

# --- Matrices ---
var A = [[1.0, 2.0], [3.0, 4.0]]
print(A @ A)            # A squared

# --- Symbolic calculus ---
sym f(x) = x**2 + 3*x
var df = d/dx(f(x))
print(df)               # 2*x + 3
print(eval(df, x=4.0)) # 11.0
```

4. The REPL — Interactive Mode

The REPL (Read-Eval-Print Loop) lets you type Vect code one line at a time and see results immediately. It is the fastest way to explore the language.

Start it by running `vect` with no arguments:

```
# Windows
venv\Scripts\vect

# macOS / Linux
venv/bin/vect
```

You will see the welcome banner and a prompt:

```
  _   _   _  
 \ \ / / _ _ | |  
  \ v / - _ | |  
   \/\_/_/_|_|
```

Vect 0.1.0 – a language with native scientific computing
Type 'help' for tips, 'exit' to quit.

```
vect>
```

Now type Vect expressions. Everything you define is remembered for the rest of the session:

```
vect> var x = 10  
vect> print(x * 3)  
30  
vect> var v = [1.0, 2.0, 3.0]  
vect> print(v + [10.0, 10.0, 10.0])  
[11, 12, 13]  
vect> sym f(x) = x**3  
vect> print(eval(d/dx(f(x)), x=2.0))  
12.0  
vect> exit  
Bye!
```

REPL special commands

Command	What it does
<code>help</code>	Show a syntax cheatsheet and available commands
<code>clear</code>	Reset the session — forget all variables and functions
<code>ir</code>	Show the LLVM IR that was generated for your last statement
<code>exit</code> or <code>quit</code>	Exit the REPL

Multi-line input in the REPL

If you open a `{` brace, the REPL knows you are writing a block and shows `...` until you close it:

```
vect> fn square(n: int) -> int {  
  ...   return n * n  
  ... }  
vect> print(square(7))  
49
```

5. Variables & Types

Declare a variable with the `var` keyword followed by a name, `=`, and a value:

```
var age      = 21           # int - a whole number  
var height   = 1.82         # float - a decimal number  
var name     = "Alice"      # string - text in double quotes  
var active   = true         # bool - true or false  
var scores   = [95.0, 87.0, 91.0] # vec - a vector of floats
```

Vect **infers the type automatically** from the value. You never need to write the type unless you want to be explicit:

```
# These are identical - annotation is optional  
var x = 42  
var x: int = 42
```

The four basic types

Type	What it holds	Examples
int	Whole numbers (64-bit)	0 , -7 , 1000000
float	Decimal numbers (64-bit)	3.14 , -0.5 , 1.5e10
bool	True or false	true , false
string	Text in double quotes	"hello" , "Vect 0.1"

And the scientific types:

Type	What it holds	Examples
<code>vec</code>	A list of float64 numbers	<code>[1.0, 2.0, 3.0]</code>
<code>mat</code>	A 2D grid of float64 numbers	<code>[[1.0,2.0],[3.0,4.0]]</code>
<code>sym</code>	A symbolic math expression	result of <code>d/dx(...)</code>

Reassigning variables

To change a variable's value after declaring it, just use `=` without `var` :

```
var count = 0
count = count + 1  # now count is 1
count = count + 1  # now count is 2
print(count)      # 2
```

IMPORTANT

Using `var` again on the same name creates a new variable and shadows the old one. Use plain `=` when you want to update an existing variable.

Type conversion

```
var n: float = float(7)    # int → float: 7.0
var m: int    = int(3.99)  # float → int: 3 (truncates)
var s: string = str(42)    # int → string: "42"
```

6. Operators & Arithmetic

Arithmetic operators

Operator	Meaning	Example	Result
<code>+</code>	Add	<code>3 + 4</code>	<code>7</code>
<code>-</code>	Subtract	<code>10 - 3</code>	<code>7</code>
<code>*</code>	Multiply	<code>4 * 5</code>	<code>20</code>
<code>/</code>	Divide	<code>10 / 4</code>	<code>2</code> (int) or <code>2.5</code> (float)
<code>%</code>	Modulo (remainder)	<code>17 % 5</code>	<code>2</code>
<code>**</code>	Power / exponentiation	<code>2.0 ** 8.0</code>	<code>256.0</code>
<code>-x</code>	Unary negation	<code>-9.8</code>	<code>-9.8</code>

Operator precedence

Operators follow standard math precedence (highest first):

Level	Operators	Associativity
1 (highest)	<code>**</code>	Right — <code>2**3**2 = 2**(3**2) = 512</code>
2	<code>* / % @ .</code>	Left
3	<code>+ -</code>	Left
4	<code>== != < <= > >=</code>	Left
5	<code>not</code>	Right
6	<code>and</code>	Left
7 (lowest)	<code>or</code>	Left

```
var a = 2 + 3 * 4      # 14 (not 20 - * before +)
var b = (2 + 3) * 4    # 20 (parentheses first)
var c = 2 ** 3 ** 2    # 512 (right-associative: 2**(3**2))
```

Comparison operators → produce bool

```
print(5 == 5)    # true
print(5 != 3)    # true
print(4 < 10)    # true
print(4 >= 4)    # true
```

Logical operators

```
var a = true
var b = false

print(a and b)   # false
print(a or b)    # true
print(not b)     # true
```

7. Control Flow

if / else if / else

```
var score = 85

if score >= 90 {
    print("Grade: A")
} else if score >= 80 {
    print("Grade: B")
} else if score >= 70 {
    print("Grade: C")
} else {
    print("Grade: F")
}

# Output: Grade: B
```

SYNTAX RULE

Curly braces { } are required around every block — even single-line ones. There is no "one-liner if" in Vect.

while loop

```
var i = 1
var total = 0

while i <= 10 {
    total = total + i
    i = i + 1
}

print(total)  # 55 — sum of 1 to 10
```

for loop — iterates over a vector

```
# Loop over a literal vector
for x in [1.0, 4.0, 9.0, 16.0] {
    print(sqrt(x))  # 1, 2, 3, 4
}

# Loop over a range of numbers
for i in range(5) {
    print(i)        # 0.0, 1.0, 2.0, 3.0, 4.0
}

# Loop over a named vector
var temperatures = [36.6, 37.1, 38.2, 36.9]
for t in temperatures {
    if t > 37.5 {
        print("Fever detected")
    }
}
```

break and continue

```
# break – exit the loop immediately
var n = 0
while true {
    n = n + 1
    if n >= 5 { break }
}
print(n)    # 5

# continue – skip to the next iteration
var i = 0
while i < 10 {
    i = i + 1
    if i % 2 == 0 { continue }
    print(i)    # 1, 3, 5, 7, 9 (skips even numbers)
}
```

8. Functions

Functions are defined with the `fn` keyword:

```
fn greet(name: string) {
    print("Hello, " + name + "!")
}
greet("Alice")    # Hello, Alice!
greet("Bob")      # Hello, Bob!
```

Functions that return a value

Use `-> type` to declare what the function returns, then use `return` :


```
fn add(a: int, b: int) -> int {
    return a + b
}

fn circle_area(r: float) -> float {
    return 3.14159 * r * r
}

print(add(10, 5))           # 15
print(circle_area(7.0))     # 153.938...
```

Recursion

Functions can call themselves. Vect handles recursion fully:

```
fn factorial(n: int) -> int {
    if n <= 1 { return 1 }
    return n * factorial(n - 1)
}

print(factorial(5))        # 120
print(factorial(10))       # 3628800

fn fib(n: int) -> int {
    if n <= 1 { return n }
    return fib(n - 1) + fib(n - 2)
}

print(fib(10))            # 55
```

Functions with local variables

```
fn hypotenuse(a: float, b: float) -> float {
    var a_sq = a * a
    var b_sq = b * b
    return sqrt(a_sq + b_sq)
}

print(hypotenuse(3.0, 4.0)) # 5.0
```

TIP — FORWARD DECLARATIONS

In Vect, you can call a function that is defined later in the file. The compiler does a first pass to register all function signatures before it compiles anything.

9. Vectors — Native First-Class Values

★ SIGNATURE FEATURE

Vectors are not objects from a library. They are a native literal type in Vect, like integers or strings. The operations are built into the language grammar.

Creating vectors

```
# A vector of floats
var v = [1.0, 2.0, 3.0]

# Assign to a variable and use later
var force    = [0.0, -9.81, 0.0]
var velocity = [10.0, 15.0, 0.0]
```

Element-wise operations

```
var a = [1.0, 2.0, 3.0]
var b = [4.0, 5.0, 6.0]

print(a + b)    # [5, 7, 9]    - element-wise add
print(a - b)    # [-3, -3, -3] - element-wise subtract
print(a * b)    # [4, 10, 18]  - element-wise multiply
```

Scalar operations

```
var v = [1.0, 2.0, 3.0]

print(v * 3.0)    # [3, 6, 9]    - scale every element
print(v * -1.0)   # [-1, -2, -3] - negate
```

Dot product — the `·` operator

The dot product multiplies corresponding elements and sums the results. It is written with the middle-dot character `·` (Unicode U+00B7).

```
var a = [1.0, 2.0, 3.0]
var b = [4.0, 5.0, 6.0]

print(a · b)    # 32.0    (1×4 + 2×5 + 3×6 = 4+10+18)
```

HOW TO TYPE `·`

Copy-paste from this document, or: on Windows press

ALT+0183

on the numpad. In VS Code with the Vect extension, it is highlighted automatically when you use it.

Indexing — read and write individual elements

```
var v = [10.0, 20.0, 30.0]

print(v[0])    # 10.0 - first element (0-based indexing)
print(v[2])    # 30.0 - last element

v[1] = 99.0    # change the second element
print(v)       # [10, 99, 30]
```

Getting the length

```
var v = [5.0, 10.0, 15.0, 20.0]
print(len(v))    # 4
```

Iterating over a vector

```
var grades = [88.0, 92.0, 74.0, 95.0]
var total  = 0.0

for g in grades {
    total = total + g
}
print(total / float(len(grades)))  # 87.25 — average
```

Real-world example — physics vectors

```
# Displacement and force vectors in 3D space
var displacement = [3.0, 4.0, 0.0]
var force        = [2.0, 0.0, 5.0]

# Work done = force · displacement
var work = force · displacement
print(work)  # 6.0 Joules

# Resultant vector
var resultant = displacement + force
print(resultant)  # [5, 4, 5]
```

10. Matrices — Native First-Class Values

★ SIGNATURE FEATURE

A matrix is written as a list of row vectors. The `@` operator is matrix multiplication, and `T()` is transpose — both native to the language.

Creating matrices

```
# A 2x2 matrix
var A = [[1.0, 2.0],
         [3.0, 4.0]]

# A 3x3 identity matrix
var I = [[1.0, 0.0, 0.0],
         [0.0, 1.0, 0.0],
         [0.0, 0.0, 1.0]]
```

Matrix multiplication — the @ operator

```
var A = [[1.0, 2.0], [3.0, 4.0]]
var B = [[5.0, 6.0], [7.0, 8.0]]

print(A @ B)
# [
#   [19, 22],
#   [43, 50]
# ]
```

Transpose

```
var A = [[1.0, 2.0, 3.0],
         [4.0, 5.0, 6.0]]

print(T(A))
# [
#   [1, 4],
#   [2, 5],
#   [3, 6]
# ]
```

Element-wise operations on matrices

```
var A = [[1.0, 2.0], [3.0, 4.0]]
var B = [[5.0, 6.0], [7.0, 8.0]]

print(A + B)      # [[6,8],[10,12]]
print(A - B)      # [[-4,-4],[-4,-4]]
print(A * 2.0)    # [[2,4],[6,8]]
```

Matrix × vector (column vector)

```
# 90-degree rotation matrix
var R = [[0.0, -1.0],
        [1.0,  0.0]]

# Column vector as a 2x1 matrix
var point = [[1.0], [0.0]]

print(R @ point)
# [
#   [0],
#   [1]
# ]
# The point (1,0) rotated 90° = (0,1)
```

Row access

```
var M = [[10.0, 20.0], [30.0, 40.0]]
var row0 = M[0]    # returns [10, 20] as a vec
print(row0)        # [10, 20]
```

11. Symbolic Differentiation — The Headline Feature

★★ THE FEATURE THAT MAKES VECT UNLIKE ANYTHING ELSE

d/dx is a real language operator. It performs symbolic calculus — computing the exact mathematical derivative of an expression, not a numerical approximation.

Step 1 — Define a symbolic function with `sym`

The `sym` keyword tells Vect that a function is symbolic — its body is a mathematical expression, not compiled code:

```
sym f(x) = x**2 + 3*x + 1
```

Step 2 — Differentiate with `d/dx`

The `d/d` prefix followed by a variable name is the differentiation operator. It returns a symbolic expression:

```
var df = d/dx(f(x))
print(df)    # 2*x + 3
```

You can use any variable name after `d/d` :

```
sym g(t) = 5*t**3 - 2*t

var dg = d/dt(g(t))
print(dg)    # 15*t**2 - 2
```

Step 3 — Evaluate numerically with `eval`

`eval(expr, var=value)` substitutes a number for the symbolic variable and returns a float:

```
sym f(x) = x**2 + 3*x + 1
var df = d/dx(f(x))    # 2*x + 3

print(eval(df, x=0.0))    # 3.0
print(eval(df, x=2.0))    # 7.0
print(eval(df, x=10.0))   # 23.0
```

Complete physics examples

Free-fall kinematics

```
# Position of a falling object under gravity
#  $s(t) = v_0 \cdot t - \frac{1}{2} \cdot g \cdot t^2$  ( $v_0=20$  m/s,  $g=9.8$  m/s2)
sym s(t) = 20.0*t - 0.5*9.8*t**2

# Velocity = ds/dt
var velocity = d/dt(s(t))
print(velocity)                # 20.0 - 9.8*t

# Velocity at t = 1 second
print(eval(velocity, t=1.0))   # 10.2 m/s

# At peak height, velocity = 0 → t ≈ 2.04s
print(eval(velocity, t=2.04))  # ≈ 0.0 m/s
```

Kinetic energy and momentum

```
#  $KE = \frac{1}{2}mv^2$  where  $m = 2$  kg
sym KE(v) = 0.5 * 2.0 * v**2

#  $dKE/dv = mv$  (this is momentum!)
var momentum = d/dv(KE(v))
print(momentum)                # 2.0*v

print(eval(momentum, v=5.0))   # 10.0 kg·m/s
print(eval(momentum, v=10.0))  # 20.0 kg·m/s
```

Polynomial curve analysis

```
#  $f(x) = x^3 - 3x^2 + 2$ 
sym f(x) = x**3 - 3*x**2 + 2

var df = d/dx(f(x))           # 3*x**2 - 6*x
print(df)

# Critical points where  $df = 0 \rightarrow x=0$  and  $x=2$ 
print(eval(df, x=0.0))        # 0.0 ← critical point
print(eval(df, x=2.0))        # 0.0 ← critical point
print(eval(df, x=1.0))        # -3.0 ← slope at x=1
```


HOW IT WORKS UNDER THE HOOD

When you write `d/dx(f(x))`, the Vect compiler converts your expression into a string that SymPy (a Python symbolic math library) can understand. SymPy computes the exact derivative. The result is stored as a symbolic expression object in Vect's runtime. When you call `eval()`, it substitutes your value and returns a float. You never see or interact with SymPy directly — it is completely hidden behind the language syntax.

12. Built-in Functions

Function	Input	Output	Example
<code>print(x)</code>	Any type	void	<code>print("hi")</code>
<code>input()</code>	—	string	<code>var s = input()</code>
<code>len(v)</code>	vec or mat	int	<code>len([1,2,3])</code> → 3
<code>sqrt(x)</code>	float	float	<code>sqrt(9.0)</code> → 3.0
<code>sin(x)</code>	float (radians)	float	<code>sin(0.0)</code> → 0.0
<code>cos(x)</code>	float (radians)	float	<code>cos(0.0)</code> → 1.0
<code>tan(x)</code>	float (radians)	float	<code>tan(0.785)</code> → ≈1.0
<code>abs(x)</code>	float	float	<code>abs(-5.5)</code> → 5.5
<code>floor(x)</code>	float	int	<code>floor(3.9)</code> → 3
<code>ceil(x)</code>	float	int	<code>ceil(3.1)</code> → 4
<code>int(x)</code>	any numeric	int	<code>int(3.7)</code> → 3
<code>float(x)</code>	any numeric	float	<code>float(7)</code> → 7.0
<code>str(x)</code>	int or float	string	<code>str(42)</code> → "42"
<code>range(n)</code>	int	vec	<code>range(4)</code> → [0,1,2,3]
<code>range(a,b)</code>	int, int	vec	<code>range(1,5)</code> → [1,2,3,4]
<code>T(m)</code>	mat	mat	<code>T(A)</code> → transpose of A
<code>eval(e,v=n)</code>	sym, binding	float	<code>eval(df, x=2.0)</code>

13. String Operations

```
# Declare and print
var s = "Hello, Vect!"
print(s)

# Concatenation with +
var first = "Ada"
var last = "Lovelace"
print(first + " " + last)    # Ada Lovelace

# Convert numbers to strings for concatenation
var score = 98
print("Score: " + str(score))    # Score: 98

# Read input from the user
print("What is your name?")
var name = input()
print("Welcome, " + name + "!")
```

Escape sequences inside strings

Sequence	Meaning
<code>\n</code>	Newline
<code>\t</code>	Tab
<code>\"</code>	Double quote inside a string
<code>\\</code>	Backslash

14. Type System & Error Messages

Vect checks types **before** your program runs. If something is wrong, you get a clear message that points at the exact line and tells you in plain English what to fix.

Undefined variable

```
print(myVar)  # forgot to declare it
```

⚠ Type Error

Type error at line 1, col 7: 'myVar' is not defined.
Check the spelling or make sure you declared it with 'var'.

Type mismatch in an operation

```
print("price: " + 9.99)
```

⚠ Type Error

Type error at line 1, col 16: Cannot use '+' between a string and a 'float'.
To concatenate, convert to string first with str(...).

Wrong number of arguments

```
fn add(a: int, b: int) -> int { return a + b }  
add(1, 2, 3)
```

⚠ Type Error

Type error at line 2, col 1: Function 'add' expects 2 argument(s) but got 3.

Type mismatch in declaration

```
var x: int = 3.14
```

⚠ Type Error

Type error at line 1, col 5: Variable 'x' is declared as 'int' but the value is a float. Use 'int(...)' to convert explicitly, or declare it as 'float'.

Type promotion — int and float mix automatically

You never need to manually convert when mixing `int` and `float` in arithmetic — Vect promotes the int to float automatically:

```
var result = 3 + 1.5    # fine - result is float 4.5
var area   = 2 * 3.14    # fine - area is float 6.28
```

15. CLI Reference

Command	What it does
<code>vect</code>	Start the interactive REPL
<code>vect run <file.vect></code>	Compile and run a program
<code>vect check <file.vect></code>	Type-check without running — find errors early
<code>vect ir <file.vect></code>	Dump the generated LLVM IR — see inside the compiler

Examples

```
# Run the demo program
venv\Scripts\vect run examples\demo.vect

# Check your code for errors before running
venv\Scripts\vect check myprogram.vect

# See the LLVM IR the compiler generated
venv\Scripts\vect ir examples\fibonacci.vect

# Start exploring interactively
venv\Scripts\vect
```

16. Complete Example Programs

Example 1 — Fibonacci sequence

```
# fibonacci.vect
fn fib(n: int) -> int {
    if n <= 1 { return n }
    return fib(n - 1) + fib(n - 2)
}

var i = 0
while i <= 10 {
    print(fib(i))
    i = i + 1
}

# Output: 0 1 1 2 3 5 8 13 21 34 55
```

Example 2 — Grade calculator

```
# grades.vect

fn letter_grade(score: float) -> string {
    if score >= 90.0 { return "A" }
    if score >= 80.0 { return "B" }
    if score >= 70.0 { return "C" }
    if score >= 60.0 { return "D" }
    return "F"
}

var scores = [95.0, 82.0, 67.0, 74.0, 55.0]
var total = 0.0

for s in scores {
    total = total + s
}

var average = total / float(len(scores))
print("Average: " + str(average))
print("Grade: " + letter_grade(average))
```

Example 3 — Linear algebra

```
# linear_algebra.vect
print("=== Dot Product ===")
var u = [3.0, 4.0, 0.0]
var v = [1.0, 0.0, 5.0]
print(u · v)    # 3.0

print("=== Matrix Multiply ===")
var A = [[2.0, 0.0], [0.0, 3.0]]    # scaling matrix
var B = [[1.0, 2.0], [3.0, 4.0]]
print(A @ B)    # [[2,4],[9,12]]

print("=== Transpose ===")
print(T(B))     # [[1,3],[2,4]]
```

Example 4 — Symbolic calculus

```
# calculus.vect
print("=== Differentiation ===")

sym f(x) = x**3 - 6*x**2 + 9*x
var df = d/dx(f(x))
print(df)                                # 3*x**2 - 12*x + 9

# Find where the slope is zero (critical points)
print(eval(df, x=1.0))                  # 0.0 ← Local max at x=1
print(eval(df, x=3.0))                  # 0.0 ← Local min at x=3

print("=== Physics ===")
sym s(t) = 30.0*t - 4.9*t**2           # throw upward at 30 m/s
var vel = d/dt(s(t))
print(vel)                             # 30.0 - 9.8*t
print(eval(vel, t=3.06))                # ≈ 0 - peak of flight
```

Example 5 — Everything together (the demo)

The file `examples/demo.vect` in the repository combines all features into one program designed to be run live. Run it with:

```
venv\Scripts\vect run examples\demo.vect
```

17. How the Compiler Works

You do not need to understand this to use Vect. But if you are curious — here is the full picture.

source.vect (text you wrote)

|

▼

LEXER	src/lexer.py
-------	--------------

Reads character by character and groups them into "tokens":

"var"	→	VAR keyword
"x"	→	IDENT identifier
"="	→	ASSIGN operator
"42"	→	INT literal
"d/dx"	→	DDIV symbolic operator

| list of tokens



PARSER	src/parser.py
--------	---------------

Reads the token list and builds a tree of nodes (the AST):
--

VarDecl("x")
└─ BinOp("+")
├─ IntLiteral(40)
└─ IntLiteral(2)

| Abstract Syntax Tree



TYPE CHECKER	src/type_checker.py
--------------	---------------------

Walks the tree and verifies types.

"x + y" where x=int, y=string → ERROR

Gives plain-English error messages pointing at the exact line/column.

| verified AST



CODE GENERATOR	src/codegen.py
----------------	----------------

Walks the tree and emits LLVM IR.

LLVM IR is a typed assembly language used by Clang, Rust, and Swift.
--

var x = 40 + 2 becomes:

%x = alloca i64

store i64 42, i64* %x

| LLVM IR text

LLVM JIT (llvmlite binding)

Compiles LLVM IR to native x86-64 machine code and runs it immediately.

running program

RUNTIME src/runtime.py

The compiled code calls Python functions (via ctypes) for:

- vector/matrix operations
- symbolic differentiation (sympy)
- print / input

The source files are all in the `src/` folder and are heavily commented. If you want to understand how any part works, open the file and read — it is written to be understood.

18. Quick Reference Card

Declarations

```
var name = value           # variable
var name: type = value     # with explicit type
fn name(p: type) -> type { ... } # function
sym name(x) = expr         # symbolic function
```

Types

```

int      42                # whole number
float    3.14              # decimal
bool     true false        # boolean
string   "hello"           # text
vec       [1.0, 2.0, 3.0]   # vector
mat       [[1.0,2.0],[3.0,4.0]] # matrix

```

Operators

```

+ - * / % **              # arithmetic
== != < <= > >=          # comparison
and or not                # logical
@                          # matrix multiply
.                          # dot product
d/dx                      # symbolic derivative

```

Control flow

```

if cond { } else if cond { } else { }
while cond { }
for x in vec { }
break continue return value

```

Symbolic calculus

```

sym f(x) = expr           # define symbolic function
var df = d/dx(f(x))       # differentiate
var n = eval(df, x=2.0)   # evaluate numerically

```

CLI commands

```

vect                # REPL
vect run <file.vect> # compile + run
vect check <file.vect> # type-check only
vect ir <file.vect>  # dump LLVM IR

```

Vect Language Guide — v0.1.0

Built by Eddie · github.com/Eddiegah/Vect

"Scientific computing should be syntax, not scaffolding."